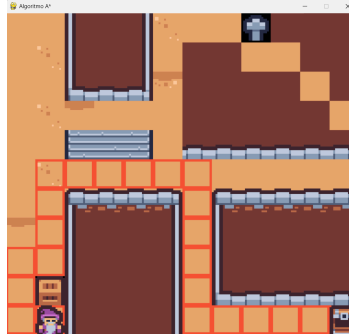


Práctica I.

Utilización de modelos de IA.

Parte I. Requisitos básicos de un sistema de resolución de problemas.



1.Resultados de aprendizaje y criterios de evaluación.

RA2: Utiliza modelos de sistemas de Inteligencia Artificial implementando sistemas de resolución de problemas.

a) Se han determinado los requisitos básicos a implementar en un sistema de resolución de problemas.

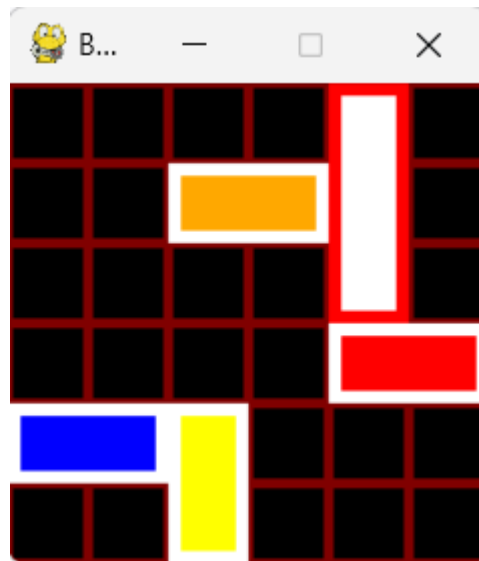
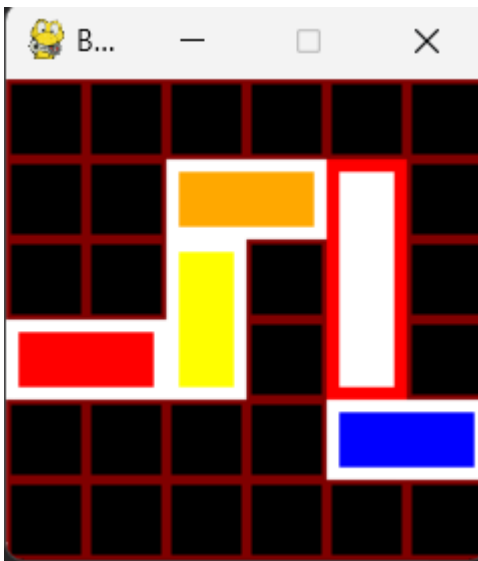
b) Se han clasificado modelos de Inteligencia Artificial.

f) Se ha valorado la adecuación de los modelos a la implementación del sistema de resolución de problemas.

2.RushHour.

RushHour es un juego en el que se han de mover vehículos horizontal y verticalmente para dejar pasar al coche rojo.

En las siguientes imágenes se puede ver la configuración inicial, y una posible solución que permite llevar el coche rojo a la parte derecha de la matriz:



Se proporciona diferentes clases que modelan el tablero y los vehículos:

Fichero [vehiculo.py](#) , contiene:

Enumeraciones, usadas en otras clases.

Orientacion.

Valores: HORIZONTAL, VERTICAL

Representa la orientación de un vehículo.

Color.

Valores: RED, GREEN, BLUE, WHITE, YELLOW, ORANGE

Métodos estáticos:

scale_color(c, factor) → Escala un color RGB multiplicando cada componente por factor.

to_rgb(color_enum) → Convierte un valor de la enumeración a su equivalente RGB como tupla (R, G, B).

Clases:

Posicion.

Atributos:

x: int → Coordenada horizontal.

y: int → Coordenada vertical.

Métodos:

copy() → Retorna una copia profunda del objeto Posicion.

Vehículo.

Atributos:

posicion: Posicion → Ubicación del vehículo.

id: int → Identificador único del vehículo.

color: Color → Color del vehículo (usando la enumeración Color).

size: int → Tamaño del vehículo (longitud en su dirección de orientación).

orientacion: Orientacion → Dirección del vehículo (horizontal o vertical).

Métodos:

__str__() → Representación en cadena del vehículo, mostrando ID, color, tamaño, posición y orientación.

copy() → Retorna una copia profunda del objeto Vehículo.

collider(other) → Detecta si el vehículo colisiona con otro Vehículo. Considera orientación horizontal/vertical de ambos vehículos. Devuelve True si hay colisión, False en caso contrario.

Fichero [tablero.py](#):

Clases:

Tablero.

Atributos:

_width: int → Ancho del tablero.

_height: int → Altura del tablero.

_scale: int → Escala de visualización (para gráficos o coordenadas).

_vehiculos: List[Vehiculo] → Lista de vehículos presentes en el tablero.

_selected: Vehiculo → Vehículo actualmente seleccionado.

_red: Vehiculo → Vehículo objetivo (rojo).

Propiedades

selected → Getter y setter para _selected.

red → Getter y setter para _red.

vehiculos → Getter para _vehiculos.

width, height, scale → Getters y setters con validación de valores positivos.

Métodos

`set_selected(x, y)` → Selecciona un vehículo basado en coordenadas.
`get_vehicle_by_position(x, y)` → Devuelve un vehículo en la posición indicada.
`move(inc_x, inc_y)` → Mueve el vehículo seleccionado.
`move_vehicle(vehiculo, inc_x, inc_y)` → Mueve un vehículo específico respetando colisiones y límites del tablero.
`move_vehicle_by_id(id, inc_x, inc_y)` → Mueve un vehículo según su ID.
`_detect_collider(vehiculo)` → Detecta si el vehículo colisiona con otro.
`eval()` → Evalúa si el vehículo rojo ha llegado al borde derecho del tablero (condición de victoria).
`backtraking()` → Implementa un algoritmo recursivo para encontrar la secuencia de movimientos que resuelve el tablero. Devuelve un vector de movimientos de tipo Nodo.

Fichero [main.py](#):

Contiene la interfaz gráfica y la interacción con el jugador, en teoría no es necesario modificar el código. **En caso de pulsar la tecla r, se vuelve al tablero original y se llama al algoritmo de backtraking** de un objeto de tipo tablero, que devuelve una lista de tuplas, cada una de ellas con [id del coche, movimiento en y, movimiento en x].

Una vez devuelta la lista, la interfaz gráfica realiza los movimientos indicados en la lista.

```

elif event.key == pygame.K_r:
    tablero.reset()
    movimientos=tablero.backtraking()

    contador=0
    pygame.time.set_timer(SIMULAR_EVENTO, INTERVALO)

```

Mapas. Se encuentran en la carpeta mapas, aunque se pueden colocar el cualquier lugar, cada fichero define un tablero en formato json (se pueden crear nuevos mapas si se desea):

```

{
  "size":{
    "width":5,
    "height":5,
    "scale":32
  },
  "vehiculos":[
    {
      "id":1,
      "color":"red",
      "posicion": {"x": 0, "y": 2},
      "orientacion":"horizontal",
      "size":2
    },
    {

```

```

        "id":2,
        "color":"yellow",
        "posicion": {"x": 3, "y": 1},
        "orientacion":"vertical",
        "size":2
    }

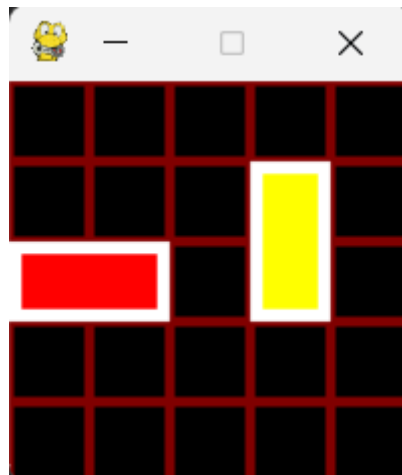
]
}

```

Cada vehículo tiene: un id, un color, una posición, una orientación y un tamaño.

Para iniciar el programa con el anterior json ejecutar:

```
python.exe .\main.py .\mapas\simple.json
```



Tarea a realizar: Implementar el algoritmo “**backtraking**” en la clase tablero, devolviendo un array/lista de movimientos para resolver el problema.

3.A*.

Implementar el algoritmo A* en un juego 2D.

Las clases que se proporcionan son:

Node: representa un nodo en el algoritmo. Destacar que con `@dataclass(order=True)` se convierte la clase en permite comparaciones de objetos (`<`, `>`, `<=`, `>=`) basados en los campos en el orden de definición, destacar que posee `f`, `g` y `h` para poder evaluar el siguiente nodo..

Mapa:

Atributos

matriz → Lista bidimensional con los valores del mapa.

matriz_mask → Lista bidimensional indicando celdas libres (1) u ocupadas (0).

inicio → Coordenadas `[x, y]` del punto de inicio.

objetivo → Coordenadas `[x, y]` del punto objetivo.

validos → Lista de valores considerados transitables. Se toman de las imágenes, `tmx` y `tsx` de la carpeta `assets`



Métodos:

`__init__()` → Inicializa la clase (actualmente vacío, idealmente inicializar listas).

`get_width()` → Devuelve el número de columnas del mapa.

`get_height()` → Devuelve el número de filas del mapa.

`load(path)` → Carga un mapa desde un CSV; establece inicio, objetivo y genera la máscara de tránsito.

`_create_matriz_mask()` → Crea `matriz_mask` a partir de `matriz`.

`get_cell(x, y)` → Devuelve el valor de la celda en `matriz`.

`get_mask_cell(x, y)` → Devuelve el valor de la celda en `matriz_mask`.

`is_wall_cell(x, y)` → Devuelve True si la celda no es transitable según válido.

`move_from_to(from, to)` → Calcula el camino de A* desde `from` hasta `to`. Devuelve un vector de posiciones `[x,y]`, que indica la siguiente celda en el camino.

Para llamar al algoritmo pulsar la tecla R, para ver el mapa en formato 0/1 pulsar la tecla D.

En este caso los mapas se han definido en formato csv, se dispone de ejemplos en la carpeta `mapas`. La primera fila indica la posición `x` e `y` del personaje y la `x` e `y` del objetivo. El resto la imagen a mostrar.

```
1 00,00,11,10
2 48,48,15,00,13,49,48,48,64,00,00,00
3 48,49,15,00,13,48,00,00,48,48,00,00
4 48,48,16,02,17,51,00,00,00,00,48,00
5 48,49,50,48,48,48,00,00,00,00,00,48
6 48,48,36,37,38,48,02,02,02,02,02,02
7 48,49,48,48,48,48,48,48,48,48,48,48
8 48,48,04,26,26,05,48,04,26,26,26,26
9 50,48,15,00,00,13,48,15,00,00,00,00
10 48,48,15,00,00,13,48,15,00,00,00,00
11 48,63,15,00,00,13,48,16,02,02,02,02
12 48,48,15,00,00,13,48,48,48,48,48,48
```

Para iniciar el programa:

```
python .\main.py .\mapas\mediano.csv
```



Tarea a realizar: Implementar el algoritmo “A*” en el método **move_from_to(from, to)** de la clase mapa, devolviendo un array/lista de movimientos para resolver el problema.

4. Evaluación.

- Backtraking: 50%.
- A*: 50%.

Cada algoritmo tiene la siguiente rúbrica:

- Implementar los métodos auxiliares: 30%.
- Resuelve el problema: 60%.
- Documenta y comenta el código y la práctica: 10%.

Si al ejecutar, da fallo de sintaxis/semántica no se corregirá.

Recordar instalar las dependencias de cada proyecto definidas en los ficheros requirements.txt

5. Entrega.

Únicamente en Aules, se fijarán el plazo en la actividad a realizar.